

1D Quadrotor Altitude Control

PID, Kalman Filter, and a Reinforcement-Learning Benchmark. Complete technical record.

Stack: Python 3.12, NumPy, Matplotlib, Gymnasium 1.3, Stable-Baselines3 2.9, PyTorch (CPU). Repository: github.com/brendancarn-y/quadrotor-pid-sim

0. Summary

This project controls the vertical altitude of a simulated quadrotor, a 1-degree-of-freedom problem (up and down only). It was built in three layers. Layer 1 is a classical PID controller driving a simple physics model. Layer 2 adds a Kalman filter so the controller acts on a clean estimate of altitude and velocity instead of a noisy sensor reading, because the PID derivative term amplifies sensor noise catastrophically. Layer 3 reframes the system as a reinforcement-learning benchmark: the plant is wrapped as a Gymnasium environment, a PPO neural-network policy is trained to control it, and it is compared head to head against the hand-tuned PID on identical dynamics, including a generalization test at altitudes the network never trained on. The result: PID wins on essentially every metric, which is the correct and expected outcome for a simple, linear, well-modeled system, and demonstrates the judgment to know when not to use machine learning.

1. The physics model (the plant)

Every layer controls the same underlying system: a point-mass drone moving vertically.

Parameters: mass $m = 1.0$ kg, gravity $g = 9.81$ m/s², timestep $dt = 0.01$ s (100 Hz simulation).

Dynamics from Newton's second law on the vertical axis. The net force is thrust minus weight, $F_{\text{net}} = T$ minus mg . Acceleration is $a = F_{\text{net}} / m = T/m$ minus g .

Integration is semi-implicit (symplectic) Euler:

```
velocity_new = velocity + a * dt
altitude_new = altitude + velocity_new * dt    # uses the NEW velocity
```

Updating velocity first, then using it to update position, is more stable than plain Euler for oscillatory systems. Defense point: the position update uses the updated velocity, which is what makes it semi-implicit.

2. Layer 1: the PID controller

A PID controller computes a corrective command from the tracking error $e = \text{target} - \text{altitude}$:

```
thrust = kP*e + kI * integral(e dt) + kD * (de/dt)
```

- **P (proportional)**: pushes in proportion to how far off you are. Alone it acts like an undamped spring, so the system oscillates forever (simple harmonic motion: nothing resists velocity, so no energy is removed).
- **D (derivative)**: responds to the rate of error change. It adds damping and stops the oscillation.
- **I (integral)**: accumulates error over time. It exists to eliminate steady-state error, a permanent offset.

The three tuned configurations (sim.py) and their results

Config	kP	kI	kD	Peak (m)	Final (m)	Overshoot
P-only	5.0	0	0	25.49	5.06	+154.9%
PD	6.0	0	4.0	8.45	8.36	-15.5%

PID	8.0	1.5	3.5	13.08	9.93	+30.8%
-----	-----	-----	-----	-------	------	--------

Target = 10 m, 15 s simulation, thrust clamped to non-negative.

Why PD settles low (about 8.37 m). In this original simulation there is no gravity feed-forward. At equilibrium the drone is motionless (velocity = 0, so the D term is 0) and there is no integral term, so the only thing producing thrust is the P term. To hover, thrust must equal weight: $k_P * e_{ss} = mg$. Solving, $e_{ss} = mg / k_P = 9.81 / 6 = 1.64$ m, so it settles at 10 minus 1.64 = 8.37 m. The steady-state error is forced by the physics, not a tuning defect.

Why PID reaches the target: the integral term keeps accumulating while any error exists, so it grows until it alone supplies the hover thrust ($k_I \text{ times integral of } e \, dt = mg$) with $e = 0$. Only then does the system stop moving.

3. Layer 2: the Kalman filter

3.1 The problem it solves

The derivative term differentiates whatever signal it is given, and a real altitude sensor is noisy. Differentiating noise makes it worse. If the measurement noise has standard deviation $\sigma_z = 0.25$ m, the numerically differentiated velocity $(y - y_{\text{prev}})/dt$ has a noise standard deviation on the order of $\sigma_z * \sqrt{2} / dt = 0.25 * 1.41 / 0.01 = \text{about } 35 \text{ m/s per step}$. Multiplied by kD , that produces violent thrust chatter.

Config	true-state chatter	measured (raw) chatter	ratio
P-only (no kD)	0.37	1.01	x3
PD ($kD=4$)	0.22	49.9	x232
PID ($kD=3.5$)	0.30	47.4	x157

Chatter = standard deviation of step-to-step thrust changes. On the raw measurement, PD final altitude blew up to 38.6 m and PID went unstable: the derivative-on-noise pathology.

3.2 How the Kalman filter works

Instead of differentiating a noisy position, velocity is estimated as part of the state. The state is $x = [z, v]$ (altitude and vertical velocity). Model matrices:

```
F = [[1, dt], [0, 1]]      # constant-velocity state transition
G = [[0.5*dt**2], [dt]]    # how acceleration enters the state
H = [[1, 0]]               # measurement model: we measure altitude only
Q = sigma_a**2 * (G @ G.T) # process noise, white-noise-acceleration model
R = [sigma_z**2] = [0.0625] # measurement-noise covariance
# sigma_a = 1.0 m/s^2 (unmodeled acceleration), sigma_z = 0.25 m
```

The two-step cycle, every timestep:

```
# Predict (project estimate forward using the model)
x_hat_minus = F @ x_hat + G * a      # a = commanded acceleration
P_minus     = F @ P @ F.T + Q

# Update (correct the prediction using new measurement y)
y_tilde = y - H @ x_hat_minus        # innovation
S        = H @ P_minus @ H.T + R     # innovation covariance
K        = P_minus @ H.T @ inv(S)     # Kalman gain
x_hat    = x_hat_minus + K @ y_tilde  # corrected estimate
P        = (I - K @ H) @ P_minus      # corrected covariance
```

The Kalman gain K is the heart of it. It decides how much to trust the new measurement versus the model prediction, weighted by their relative uncertainties. A noisy sensor (large R) pulls K down and leans on the model; an unreliable model (large Q) pushes K up toward the sensor. It rebalances every step. Defense point: K is not a fixed constant, it is recomputed each step from the covariances.

3.3 The biggest issue: $a = 0$ versus commanded acceleration

The textbook starter predicts with acceleration $a = 0$ (a pure constant-velocity assumption, letting the update step catch up). Built that way first, it destabilized the control loop: with $a = 0$ the filter cannot see the drone accelerating, so during the hard thrust of a climb the velocity estimate lagged far behind reality. In the unbounded-thrust simulation, P-only got pumped past 130 m (should peak about 25 m) and PD and PID developed large biases. The estimate lag created a phase lag in the feedback loop, which destabilized it. The fix: feed the commanded acceleration $a = (T - \text{mg})/m$ into the predict step.

Config	a=0 peak	commanded-accel peak	true-state peak
P-only	129.2	25.3	25.5
PD	81.5	8.48	8.45
PID	112.9	12.75	13.08

This is the single most important thing to defend. The lesson: a Kalman filter predict step should use the known control input. Ignoring it is only acceptable for gently-moving systems. Once applied, the filtered controller matched true-state performance almost exactly.

3.4 A second subtlety: the derivative refactor

The original PID computed its derivative as $(\text{error} - \text{previous_error})/dt$. With a constant setpoint, $d(\text{error})/dt = -\text{velocity}$. So once a velocity estimate existed from the filter, the controller was refactored to take the derivative term as $-\hat{v}$ directly, rather than re-differentiating a signal. This is cleaner and avoids re-introducing noise. On clean data it reproduces the original derivative exactly: the refactored controller gave bit-identical results on true state.

4. Layer 3: the reinforcement-learning benchmark

4.1 The purpose

This is not an RL project and the goal is not for the network to win. The deliverable is one benchmarked comparison and one result sentence with a number in it. The point is to demonstrate you can wrap a control problem as a standard RL environment, train a modern policy-gradient algorithm on it, and run a fair benchmark and read the result honestly. For a simple, linear, low-dimensional, well-modeled plant, classical control (PID) is already near-optimal. A correct negative result demonstrates more understanding than pretending the ML succeeded. Learned control earns its keep on nonlinear, high-degree-of-freedom, or hard-to-model systems.

4.2 The Gymnasium environment (`envs/altitude_env.py`)

`step(action)` advances the physics one tick and returns (observation, reward, terminated, truncated, info), letting any controller drive the identical plant.

- **Action space:** normalized thrust a in $[-1, 1]$, mapped by $T = \text{hover} + a \cdot (T_{\text{max}} - \text{hover})$, where $\text{hover} = mg = 9.81 \text{ N}$ and $T_{\text{max}} = 2mg = 19.62 \text{ N}$. So $a = 0$ gives exactly hover thrust and $a = \pm 1$ spans a thrust-to-weight ratio of 2. Gravity is now fed forward by the hover offset, and the actuator has a realistic upper limit (the original sim had unbounded thrust).
- **Observation:** the Kalman-filtered estimate $[\text{target} - \hat{z}, \hat{v}]$. Both controllers see this identical clean estimate.
- **Reward:** $-(e_{\text{true}} / z_{\text{target}})^2 - 0.01 \cdot a^2$. The squared-error term is normalized by the target so the reward scale stays sane (reward scaling is the top cause of failed training). The effort term stops motor-slamming. Reward and termination use the TRUE state, privileged information available only during training and eval; the controller only ever sees the filtered estimate.
- **Termination:** episode ends if $|z| > 3 \cdot z_{\text{target}}$ or after $\text{max_steps} = 1000$ ticks (10 s).

4.3 The Kalman filter as a shared front-end (fairness)

The PPO policy used here (`MlpPolicy`) is a plain feed-forward network: memoryless, seeing only the current observation. A memoryless network cannot filter noise over time, because filtering is inherently temporal. Fed the raw noisy sensor, it would be structurally incapable of what the Kalman filter does, and the comparison would be unfair. Solution: put the Kalman filter in front of both controllers. Both receive the identical clean estimate. The filter is shared infrastructure, a perception layer, not part of the contest. This makes the comparison purely about the control law and reuses the Layer-2 work coherently.

4.4 The PID as a policy object (`controllers/pid.py`)

The PID is repackaged as a class with an `act(obs)` method, the same interface the RL policy has. The control law is unchanged. It reads error and velocity from the observation, uses $-\hat{v}$ as the derivative term, and clips its output to $[-1, 1]$. Re-tuning was required: the original gains (raw-thrust units) do not transfer to the normalized action space. Re-tuned to $kP = 1.5$, $kD = 0.8$, $kI = 0$.

- **Why $kI = 0$:** the hover offset already feeds gravity forward, so a PD controller reaches zero steady-state error on its own (at the target, $a = 0$ holds hover with $e = 0$). An integral term only hurts: it winds up during the climb and biases the result. Verified empirically.
- **Result:** the re-tuned PD tracks 10 m at 0.03 m steady-state error, 0.8% overshoot, 1.8 s settling, stable across seeds.

4.5 Training the network (`train_ppo.py`)

- **Algorithm:** PPO (Proximal Policy Optimization), an on-policy actor-critic method, via `Stable-Baselines3` with the default `MlpPolicy`.

- Budget: 300,000 timesteps, seed = 0, trained only at the 10 m setpoint.
- `check_env()` validated the environment against the Gymnasium contract before training (caught nothing).
- Mean episode reward plateaued around minus 80.8. Context: doing nothing scores about minus 1000; near-perfect tracking about minus 15 to minus 40. The network learned a competent controller, measurably worse than PID.

4.6 The evaluation harness (evaluate.py)

Runs both controllers through identical episodes and records the true altitude (env.z, not the estimate). Metrics, each averaged over 10 seeds: steady-state error = |setpoint minus mean(last 50 samples)|; overshoot = max(0, (peak minus setpoint)/setpoint * 100%); settling time = the last time the response leaves the plus-or-minus 2% band. Generalization test: the network was trained only at 10 m, then both controllers are evaluated at 6 m and 14 m, setpoints the network never saw.

4.7 The results

Setpoint	Controller	Steady-state err (m)	Overshoot	Settling (s)
10 m (trained)	PID	0.03	0.9%	1.84
	PPO	0.10	9.1%	2.77
6 m (unseen)	PID	0.03	1.3%	2.03
	PPO	0.10	1.1%	8.54
14 m (unseen)	PID	0.03	9.1%	3.32
	PPO	0.10	17.4%	3.55

Result sentence: PID beat the PPO policy on every metric at the trained 10 m setpoint (steady-state error 0.03 vs 0.10 m, overshoot 0.9% vs 9.1%, settling 1.8 vs 2.8 s), and generalized better to unseen setpoints, the expected outcome for a linear, low-dimensional plant where classical control is already near-optimal.

Interpretation for defense:

- PID steady-state error is identical (0.03 m) at every setpoint because it is a fixed control law independent of the target magnitude.
- PID overshoot grows at 14 m (9.1%) because the larger initial error saturates the actuator ($a = 1$, full thrust) for longer, building extra momentum. Expected actuator-saturation behavior, not a tuning flaw.
- The network generalizes worse: it carries a fixed about 0.1 m steady-state offset learned at 10 m. At 6 m that offset sits near the tighter 2% band (0.12 m), so noise repeatedly nudges it across the boundary and the settling detector fires late (8.5 s). At 14 m it extrapolates beyond its training range and overshoots more (17.4%).

5. Every issue worked out (chronological)

- 1 **Kalman $a = 0$ destabilized the loop.** Fixed by feeding commanded acceleration into the predict step. The headline technical issue.
- 2 **Derivative amplified noise** (the entire motivation for Layer 2). Estimate velocity in the state instead of differentiating position.
- 3 **PID derivative refactor.** Take the derivative as minus v_{hat} from the filter rather than re-differencing error.
- 4 **Gains did not transfer to the normalized action space.** Re-tuned to $kP=1.5$, $kD=0.8$, and found the integral term unnecessary and harmful under gravity feed-forward.
- 5 **Reward scale.** Normalized the squared-error term by the target so training does not see rewards near plus or minus 100.
- 6 **Termination-bound bug.** The left-the-world limit was hardcoded to $|z| > 10$, which at a 10 m target ends the episode the instant you arrive; raised to $3 \cdot z_{\text{target}}$.

- 7 **Memoryless network cannot filter noise.** Placed the Kalman filter as a shared front-end so the comparison stays fair.
- 8 **Scripts failed from the wrong folder.** Anchored all file paths to the script location via `__file__` so model loading and figure saving always resolve.
- 9 **ModuleNotFoundError in VS Code.** Libraries were in the project `.venv` but the Run button used global Python; installed the dependencies into the global interpreter too and documented the `venv` path.
- 10 **Script looked frozen.** PyTorch takes about 4 s to import and the full run about 19 s with no early output; added progress messages so it is not mistaken for a hang.
- 11 **Repository was messy.** Two figures both named `comparison.png`, a stray `plot.png`, no dependency manifest; reorganized outputs under `results/` with distinct names, added `requirements.txt` and a one-click `run_benchmark.bat`.
- 12 **Version-control hygiene.** Cleaned an AI co-author trailer from commit history (history rewrite plus force-push) and updated the git remote after the GitHub username changed from `brendanacarney-debug` to `brendancarn-y`.

6. Anticipated defense questions

Why does PD settle below target in the first simulation but not in the RL environment? The first sim has no gravity feed-forward, so PD needs a steady error to generate hover thrust ($e_{ss} = mg/kP = \text{about } 1.64 \text{ m}$). The RL environment feeds gravity forward via the hover offset, so $a = 0$ holds hover and PD reaches zero steady-state error.

Why feed commanded acceleration into the Kalman predict step? With $a = 0$ the filter cannot see the drone accelerating, so the estimate lags during climbs, creating a phase lag that destabilized the loop (P-only hit 130 m). Feeding the known control input fixes it.

Is it unfair that the network is memoryless and cannot filter? That is exactly why the Kalman filter is a shared front-end feeding both controllers the same clean estimate. The benchmark compares control laws, not perception.

Why did the network lose? Is ML not supposed to be better? For a linear, low-dimensional, well-modeled plant, PID is near-optimal; there is no nonlinearity or coupling for the network to exploit. ML wins on hard-to-model or high-DOF systems. Knowing the distinction is the point.

How do you know the physics is the same across all three layers? The environment dynamics are copied verbatim from the original simulation (same semi-implicit Euler, same $a = T/m$ minus g), and a regression check confirmed the PID tracks sensibly after the move.

Why a 2% settling band and why average over 10 seeds? 2% is the standard control settling criterion; averaging over seeds smooths the random sensor noise so the metrics are repeatable.

What is $Q = \sigma_a^2 * G * G\text{-transpose}$? The standard discrete white-noise-acceleration process-noise model: uncertainty enters as an unknown acceleration of standard deviation σ_a , propagated into position and velocity through G .

7. Limitations and honest caveats

- The task is deliberately favorable to PID. This is a controlled baseline, not evidence that RL is generally worse.
- The network was trained once with default hyperparameters and a simple reward, not exhaustively tuned. Chasing a win was declined; the honest negative result is the deliverable.
- The Kalman motion model is a slight approximation of the exact integration, but the process-noise term Q absorbs the mismatch and it is negligible at $dt = 0.01$.
- Plots use a single representative seed; metrics are seed-averaged.
- 1-DOF only, no horizontal motion or attitude. Extending to 6-DOF is where the RL-versus-PID comparison gets genuinely interesting, and where PID would start to struggle.

8. Possible future work

- Feed the commanded acceleration to the network too (it currently only gets the position and velocity estimate).
- A noisy-observation variant with a recurrent (LSTM) policy that must learn its own filtering.
- Extend to 2-DOF or full 6-DOF, where nonlinear coupling could let the learned policy actually win.

9. How to run

```
python -m venv .venv
.venv\Scripts\activate          # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
python train_ppo.py             # trains PPO, saves ppo_altitude.zip (~4 min, CPU)
python evaluate.py              # writes results/comparison.png + metrics table
# Windows: double-click run_benchmark.bat to run and open the figure
# python sim.py produces the standalone Layer-1/2 figure
```

10. Repository structure

```
quadrotor-pid-sim/
  README.md          # leads with the PID-vs-PPO result
  requirements.txt
  run_benchmark.bat  # one-click run + open figure (Windows)
  sim.py             # Layers 1-2: PID + Kalman standalone simulation
  train_ppo.py       # trains the PPO policy
  evaluate.py        # PID-vs-PPO benchmark + generalization test
  envs/altitude_env.py # the plant as a Gymnasium environment
  controllers/pid.py # PID as a policy object
  results/
    comparison.png    # PID vs PPO (headline figure)
    kalman_comparison.png # PID + Kalman (foundation figure)
```